Informe de Sistema distribuido para servicios de computación de alto rendimiento (HPC) – UPB-CIENTIFICA

Andrés Felipe Pinzón Camacho[1], Sergio Alejandro Sanabria Moreno[1],

Angie Lorena Villareal Vásquez[1], Valentina Téllez Salazar[1],

Fernando Cancelado Figueroa[1] y Daniel Felipe Aguilar Latorre[1]

Universidad Pontificia Bolivariana Seccional Bucaramanga, COLOMBIA

Resumen. Este artículo presenta el diseño de un sistema distribuido orientado a dotar a la microempresa UPB-CIENTÍFICA de servicios de computación de alto rendimiento (HPC) y de gestión colaborativa de archivos, soportado en una infraestructura de cómputo en malla (Grid) alojada en el Centro de Computación Avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga. El sistema integra cinco servicios —ejecución de trabajos MPI, sincronización de archivos, compartición de archivos mediante RMI, álbum de fotografías y streaming mediante SOAP— bajo un esquema común de autenticación centralizada, seguridad informática y monitoreo. Se presenta el estado del arte que sustenta las decisiones tecnológicas del proyecto, el marco conceptual, los objetivos, la metodología ágil adoptada y el cronograma planeado para su desarrollo durante el periodo académico 2026-20.

Palabras clave: sistemas distribuidos, computación de alto rendimiento, computación en malla, RMI, MPI.

1 Introducción

La computación de alto rendimiento (HPC) se ha convertido en un recurso estratégico para la comunidad científica, al permitir el procesamiento de grandes volúmenes de datos y la ejecución de simulaciones complejas en tiempos razonables. Sin embargo, poner esta capacidad de cómputo al servicio de investigadores externos exige mucho más que un clúster de máquinas: requiere una infraestructura distribuida capaz de administrar el acceso, almacenar y sincronizar los datos de trabajo, compartir resultados de forma segura y monitorear continuamente su disponibilidad.

Este artículo presenta el diseño de un sistema distribuido para la microempresa UPB-CIENTÍFICA, dedicada a ofrecer servicios de HPC a la comunidad científica mediante el procesamiento paralelo de código basado en MPI. Como parte del proyecto de aula de la asignatura Sistemas Distribuidos (periodo 2026-02) de la Universidad Pontificia Bolivariana seccional Bucaramanga, el sistema propuesto integra una infraestructura

de cómputo en malla (Grid) con cuatro servicios complementarios de gestión de archivos —sincronización, compartición, álbum de fotografías y streaming—, todos alojados en el Centro de Computación Avanzada de la universidad y desarrollados exclusivamente con software libre. El documento se organiza de la siguiente manera: la sección 2 presenta el estado del arte que sustenta las decisiones tecnológicas del proyecto; la sección 3 expone el marco conceptual; la sección 4 detalla los objetivos generales y específicos; la sección 5 describe la metodología de desarrollo adoptada; las secciones 6 y 7 presentan los resultados obtenidos y el cronograma ejecutado hasta la fecha; y la sección 8 cierra con la discusión y las conclusiones del trabajo.

2 Estado del arte

A continuación, se presentan los ejes temáticos clave que sustentan el diseño técnico del proyecto, organizados según los componentes tecnológicos exigidos por la situación problemática de UPB-CIENTÍFICA: computación en malla, middleware de objetos distribuidos, protocolos de comunicación entre servicios, sincronización y compartición de archivos, streaming de video, autenticación centralizada, seguridad, monitoreo de infraestructura HPC, cómputo paralelo con MPI y tolerancia a fallos.

2.1 Computación en malla y su comparación con la nube

Adepoju y Ologunagba (2025) presentan una revisión sistemática que compara la evolución de la computación en malla frente a las infraestructuras modernas de computación en la nube, examinando arquitecturas, aplicaciones y tendencias emergentes de ambos paradigmas. Los autores destacan que, si bien la nube ofrece elasticidad y facilidad de aprovisionamiento, la computación en malla conserva ventajas en escenarios donde se requiere integrar recursos heterogéneos y geográficamente distribuidos —como computadores, dispositivos móviles y equipos de laboratorio— bajo un esquema de propiedad compartida entre instituciones, en lugar de depender de un proveedor comercial único (Adepoju & Ologunagba, 2025). En un sentido similar, Sungkar y Kogoya (2020) describen la computación Grid como un tipo de sistema paralelo y distribuido que permite compartir, seleccionar y agregar recursos autónomos dispersos geográficamente en tiempo de ejecución, según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario. Para el proyecto de UPB-CIENTÍFICA, estos hallazgos son directamente aplicables: la exigencia de integrar computadoras, dispositivos móviles e IoT al clúster HPC, tanto interna como externamente, corresponde exactamente al modelo de computación en malla (Grid) descrito en ambos estudios, y no a un esquema de nube pública, que además está descartado por las restricciones presupuestales y de infraestructura del proyecto.

2.2 Middleware de objetos distribuidos: RMI y CORBA

La selección de RMI como tecnología obligatoria para el servicio Shared File exige comprender sus fortalezas y limitaciones frente a alternativas como CORBA. Munoz y Zalewski (2001) realizan una comparación arquitectónica y de desempeño entre RMI y

CORBA sobre plataformas Solaris y Windows NT, midiendo el tiempo de respuesta ante invocaciones de métodos remotos con distinto número de parámetros y bajo carga de múltiples clientes concurrentes; sus resultados ofrecen un marco de referencia para decidir entre ambos mecanismos en aplicaciones distribuidas en tiempo real. Juric (2000) complementa este análisis con una comparación cualitativa y cuantitativa centrada en la facilidad de desarrollo, la madurez y el desempeño de RMI frente a CORBA en aplicaciones basadas en Java, concluyendo que RMI resulta más sencillo de integrar en entornos homogéneos Java, mientras que CORBA maneja mejor la interoperabilidad multilenguaje y la carga de múltiples clientes simultáneos, aunque con mayor sobrecarga. Estos hallazgos son relevantes para justificar, dentro de la arquitectura de UPB-CIENTÍFICA, el uso de un servidor RMI en clúster para el servicio de archivos compartidos, dado que el ecosistema de la aplicación está mayoritariamente basado en Java y no requiere interoperabilidad con otros lenguajes en ese componente específico.

2.3 Protocolos de comunicación entre servicios: SOAP, REST y gRPC

El sistema de UPB-CIENTÍFICA integra explícitamente tres estilos de comunicación entre servicios: SOAP (para el servicio de streaming), un servidor web basado en JavaScript y gRPC (para los trabajos MPI), lo que exige comprender sus diferencias de diseño. Castillo, Bernier, Arenas, Merelo y García-Sánchez (2011) comparan experimentalmente SOAP y REST implementando un algoritmo genético maestro-esclavo sobre ambos protocolos, encontrando diferencias de tiempo significativas: SOAP resulta conceptualmente más complejo y pesado, pero ofrece un contrato de interfaz más formal (WSDL), mientras que REST resulta más liviano para escenarios de alta frecuencia de solicitudes. Por su parte, la literatura técnica reciente coincide en que gRPC, al basarse en HTTP/2 y en la serialización binaria de Protocol Buffers, ofrece menor latencia y mayor throughput que SOAP o REST en comunicación interna entre microservicios, siendo especialmente adecuado para operaciones de streaming bidireccional, lo que explica su elección como tecnología para el servicio de ejecución de trabajos MPI sobre el clúster HPC, un componente donde el rendimiento de la comunicación es crítico.

2.4 Sincronización y compartición de archivos distribuidos

El servicio File Sync de UPB-CIENTÍFICA requiere sincronizar automáticamente los árboles de directorios de los dispositivos cliente con un repositorio central, manteniendo consistencia entre todas las copias. Mehdi y Ragab-Hassen (2016) presentan una encuesta de tecnologías de sincronización de archivos, tanto centralizadas en la nube (Dropbox, Google Drive) como basadas en P2P, discutiendo sus fortalezas y debilidades en términos de ancho de banda, latencia y consistencia. Los autores destacan que los protocolos de sincronización delta —que solo transmiten los cambios (deltas) entre versiones de un archivo, en lugar del archivo completo— reducen significativamente el consumo de red, un principio de diseño aplicable directamente al servicio de backup síncrono exigido por el proyecto. Complementariamente, el estudio cualitativo de Capra (2014), basado en entrevistas a usuarios universitarios, identifica limitaciones prácticas percibidas por los usuarios en sistemas de sincronización automática, como la

imposibilidad de sincronizar carpetas parcialmente o la necesidad de procesos manuales de respaldo en algunos escenarios, lo que resulta relevante para el diseño de la experiencia de usuario del servicio Home parametrizado que exige UPB-CIENTÍFICA para cada cliente.

2.5 Arquitecturas de streaming de video adaptativo

El servicio de Streaming de UPB-CIENTÍFICA debe ofrecer reproducción fluida, control de acceso y gestión de permisos para videos de presentaciones científicas. Natali y Merani (2015) proponen una arquitectura de streaming adaptativo (P2P-DASH) que combina el estándar Dynamic Adaptive Streaming over HTTP (DASH) con redes peer-to-peer, implementando una estrategia de control de tasa distribuida que ajusta la calidad del video según las condiciones cambiantes de la red, sin depender exclusivamente de un servidor central. En una línea similar, Waheed et al. (2021) presentan una arquitectura eficiente para streaming de video en tasa de bits adaptativa (ABR), evaluando el tiempo de respuesta y el throughput frente a distintos tamaños de segmento de video, y demostrando que el consumo de CPU y energía puede optimizarse ajustando estos parámetros sin sacrificar la calidad percibida por el usuario. Ambos estudios son relevantes para el diseño del servicio de streaming SOAP de UPB-CIENTÍFICA, en la medida en que sugieren estrategias para equilibrar la calidad de reproducción y el consumo de recursos del servidor, algo especialmente importante dado que la infraestructura debe alojarse íntegramente en el Centro de Computación Avanzada de la UPB sin apoyo de redes de distribución de contenido externas.

2.6 Autenticación centralizada y directorios activos

UPB-CIENTÍFICA exige que los perfiles de usuario —autenticación, autorización, mapa de servicios y árboles de archivos— sean administrados por un servicio de directorio activo. El trabajo clásico de Woo y Lam (1992) sienta las bases teóricas de la autenticación en sistemas distribuidos, formalizando los requisitos de un protocolo de autenticación seguro frente a ataques de intermediario y repetición, principios que continúan vigentes en los protocolos modernos usados por servicios de directorio como LDAP y Active Directory. Sobre esa base, la literatura técnica reciente coincide en que LDAP y Active Directory siguen siendo las tecnologías fundacionales para la gestión centralizada de identidades, permitiendo unificar la autenticación, el control de acceso basado en roles y la sincronización de directorios entre entornos heterogéneos, tanto locales como híbridos. Para el diseño de UPB-CIENTÍFICA, esto sustenta la decisión de emplear un servicio de directorio activo como fuente única de verdad para los perfiles de usuario compartidos entre los cinco servicios del sistema (File Sync, Shared File, Photo Album, Streaming y MPI Jobs), evitando la duplicación de credenciales y facilitando la trazabilidad de accesos exigida por los requisitos de seguridad del proyecto.

2.7 Seguridad en sistemas distribuidos

El proyecto exige cifrado asimétrico GPG para las copias de seguridad, autenticación basada en token o múltiple factor, y un análisis de vulnerabilidades sobre el ambiente

de producción. Priyadarshini (2022) ofrece una perspectiva de revisión sobre la seguridad en sistemas distribuidos, dividiendo el problema en dos frentes: la seguridad de la comunicación entre usuarios y procesos —autenticación, integridad de mensajes y cifrado— y el control de los permisos de acceso a los recursos, incluyendo el caso particular del código móvil. En una línea más aplicada, Babu, Abraham y Borasia (2013) revisan la criptografía de clave simétrica como mecanismo de aseguramiento en sistemas distribuidos, comparando algoritmos como DES y AES, y enumerando los pilares de la seguridad distribuida: autenticación, confidencialidad, autorización, integridad, disponibilidad y trazabilidad (accountability). Estos marcos conceptuales son directamente aplicables al diseño de seguridad de UPB-CIENTÍFICA, en el que conviven mecanismos de cifrado asimétrico para las copias de seguridad (confidencialidad de datos en reposo) con controles de autenticación multifactor (verificación de identidad), evidenciando que ambos frentes deben abordarse de forma complementaria y no aislada.

2.8 Monitoreo de clústeres e infraestructuras HPC

El módulo de monitoreo exigido por UPB-CIENTÍFICA debe reportar en tiempo real la ocupación de recursos, la carga de trabajos y la disponibilidad de servicios, con históricos y alertas configurables. Kunz (2022), en su tesis sobre monitoreo de trabajos HPC, implementa una solución basada en el gestor de recursos SLURM junto con Prometheus, para la recolección de métricas, y Grafana, para su visualización, evaluando enfoques previos como los marcos PIKA y ClusterCockpit que además permiten visualizar la distribución topológica de los procesos sobre los nodos del clúster. La documentación técnica de referencia coincide en que este mismo patrón —exportadores de métricas por nodo alimentando una base de series temporales, con paneles de visualización y alertamiento configurable— constituye el enfoque estándar de facto para el monitoreo de clústeres de cómputo de alto rendimiento en entornos on-premise, similares al que exige alojar UPB-CIENTÍFICA dentro del Centro de Computación Avanzada de la UPB. Esto sustenta la elección de una pila de monitoreo basada en herramientas de código abierto para cumplir con el requisito de licenciamiento libre del proyecto, evitando depender de soluciones de monitoreo propietarias o basadas en la nube.

2.9 MPI y paradigmas de cómputo paralelo

UPB-CIENTÍFICA se especializa en el procesamiento de código en paralelo mediante MPI (Message Passing Interface) sobre su clúster HPC, por lo que resulta central comprender su rol frente a otros paradigmas de paralelismo. AlHafez y Kurdi (2025) realizan un análisis comparativo de los tres paradigmas dominantes en HPC moderno —MPI, OpenMP y CUDA— evaluando sus fundamentos arquitectónicos, características de desempeño e idoneidad según el dominio de aplicación. Sus resultados muestran que MPI sobresale en entornos de memoria distribuida, logrando una escalabilidad casi lineal en aplicaciones intensivas en comunicación entre nodos, aunque enfrenta desafíos de sobrecarga de comunicación a medida que crece el número de procesos, mientras que OpenMP resulta más adecuado para el paralelismo de memoria compartida dentro de un mismo nodo. Esta distinción es relevante para el diseño del entorno de administración y ejecución de trabajos científicos de UPB-CIENTÍFICA, ya que confirma que

MPI es la tecnología correcta para coordinar el procesamiento distribuido entre los distintos nodos del clúster de la infraestructura Grid, reservando paradigmas de memoria compartida para la optimización interna de cada nodo si fuese necesario en fases posteriores del proyecto.

2.10 Tolerancia a fallos y replicación

Aunque el proyecto de UPB-CIENTÍFICA no detalla explícitamente mecanismos de replicación, el syllabus del curso incluye la tolerancia a fallos y la alta disponibilidad como aspectos críticos transversales a cualquier sistema distribuido. El trabajo tutorial de Guerraoui y Schiper (1996), aunque clásico, continúa siendo una referencia estructural en este campo: los autores definen la linealizabilidad como criterio de corrección para servicios replicados y presentan las dos grandes familias de técnicas de replicación —replicación primario-respaldo (primary-backup) y replicación activa—, introduciendo la comunicación de grupo como infraestructura necesaria para mantener la consistencia entre réplicas ante fallos parciales del sistema. Estos conceptos resultan pertinentes para UPB-CIENTÍFICA en al menos dos frentes: por un lado, el repositorio distribuido de archivos, fotos y videos podría beneficiarse de un esquema de réplicas para garantizar disponibilidad ante la caída de un nodo del Centro de Computación Avanzada; por otro, el propio clúster HPC, al ejecutar trabajos científicos de larga duración, requiere mecanismos de checkpointing y recuperación ante fallos que se inspiran directamente en los principios de tolerancia a fallos por replicación descritos por los autores.

3 Marco conceptual

A continuación, se definen los conceptos técnicos centrales sobre los cuales se apoya el diseño del sistema, con el fin de fijar un vocabulario común para las secciones posteriores de este artículo.

3.1 Sistema distribuido

Un sistema distribuido está compuesto por múltiples nodos autónomos que se comunican por red y se presentan ante el usuario como un único sistema coherente, coordinando su funcionamiento sin depender de un reloj o una memoria compartida.

3.2 Computación en malla (Grid computing)

Es el modelo de sistema distribuido en el que se comparten, seleccionan y agregan recursos autónomos y geográficamente dispersos —computadoras, dispositivos móviles y elementos de IoT— según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario, sin depender de un proveedor comercial único como ocurre en la nube pública.

3.3 Computación de alto rendimiento (HPC)

Es el subconjunto de la computación distribuida orientado a maximizar la capacidad de procesamiento mediante clústeres de múltiples nodos, típicamente empleados para ejecutar trabajos científicos, de ingeniería o de análisis de datos a gran escala.

3.4 MPI

Es la interfaz de paso de mensajes (Message Passing Interface) que permite coordinar la ejecución de un mismo programa entre múltiples procesos que no comparten memoria, propia de los paradigmas de memoria distribuida sobre clústeres HPC.

3.5 Middleware de objetos distribuidos

Son las capas de software (RMI en Java, CORBA de forma multilenguaje) que permiten invocar métodos de objetos ubicados en procesos remotos como si fueran locales, ocultándole al programador los detalles de serialización y comunicación de red.

3.6 Servicios web (SOAP, REST y gRPC)

Son los estilos de comunicación entre servicios sobre los cuales se apoya el proyecto: SOAP, basado en XML y con un contrato formal de interfaz (WSDL); REST, un estilo arquitectónico liviano orientado a recursos sobre HTTP; y gRPC, basado en HTTP/2 y en la serialización binaria de Protocol Buffers, orientado a comunicación interna de baja latencia entre servicios.

3.7 Sincronización de archivos

Es el mecanismo mediante el cual los cambios realizados en el árbol de directorios de un dispositivo cliente se propagan automáticamente hacia un repositorio central, manteniendo consistencia entre todas las copias del recurso.

3.8 Directorio activo

Es el servicio centralizado (LDAP / Active Directory) que administra los perfiles de usuario de un sistema distribuido, unificando la autenticación, la autorización y el control de acceso basado en roles entre los distintos servicios que lo componen.

3.9 Seguridad informática distribuida

Es el conjunto de mecanismos —cifrado, autenticación, integridad, disponibilidad y trazabilidad— orientados a proteger la comunicación entre procesos y el acceso a los recursos de un sistema distribuido.

3.10 Monitoreo de infraestructura

Es la recolección continua de métricas de uso (CPU, memoria, almacenamiento, carga de trabajos y disponibilidad) sobre los nodos de una infraestructura, típicamente mediante exportadores por nodo, una base de series temporales y paneles de visualización y alertamiento.

4 Objetivos

4.1 Objetivo general

Desarrollar un sistema distribuido que proporcione servicios de computación de alto rendimiento (HPC) y gestión colaborativa de archivos, mediante una infraestructura en malla alojada en el centro de computación avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga, para permitir a la microempresa UPB-CIENTIFICA y a la comunidad científica ejecutar aplicaciones de procesamiento intensivo de manera segura y escalable, así como administrar, compartir recursos y datos científicos de forma centralizada.

4.2 Objetivos específicos

1. Documentar los requerimientos funcionales y no funcionales de la computación en malla, computación de alto rendimiento y los paradigmas de comunicación distribuida pertinentes, con el fin de definir la arquitectura general del entorno de administración y ejecución de trabajos paralelos en MPI.
2. Desarrollar el servicio de sincronización de archivos que garantice consistencia entre los dispositivos cliente y el repositorio central, incluyendo la sincronización al detectar conexión a la red interna.
3. Desarrollar el servicio de compartir archivos mediante Java RMI, respetando el modelo de permisos Unix sobre un espacio parametrizado y con cuota máxima por usuario.
4. Desarrollar el servicio de álbum de fotografías y el servicio de streaming mediante SOAP con funciones de ordenamiento, control de acceso y gestión de permisos para los usuarios autorizados del sistema.
5. Integrar los distintos componentes del sistema mediante los métodos de comunicación especificadas para el proyecto, garantizando la interoperabilidad entre los servicios desplegados en los distintos servidores.
6. Desarrollar un servicio de directorio activo para la administración centralizada de los perfiles de usuario, incluyendo autenticación, autorización, mapa de servicios y arboles de archivos.
7. Implementar mecanismos de seguridad informática mediante cifrado asimétrico GPG para las copias de seguridad, autenticación basada en token y análisis de vulnerabilidades a la solución tecnológica desarrollada.

8. Implementar un módulo de monitoreo integral, accesible vía web, que reporten en tiempo real la ocupación de recursos, la carga de trabajos y la disponibilidad de los servicios del sistema, con históricos y alertas configurables.

5 Metodología

El proyecto adopta un modelo de desarrollo iterativo e incremental (Sommerville, 2011) como base metodológica, en el que el ciclo de vida del software —análisis, diseño, implementación, pruebas e integración— se recorre de manera completa en cada iteración, en lugar de ejecutarse una sola vez a lo largo de todo el proyecto. Esta elección responde a las particularidades de un proyecto de infraestructura distribuida con múltiples servicios desarrollados en paralelo por sub-equipos: permite que el equipo, de seis integrantes, entregue en cada iteración un incremento funcional y verificable del sistema, alineado con las entregas incrementales que exige la asignatura (bitácoras en las semanas 8 y 16, producto en la semana 14).

El desarrollo se organiza en cuatro iteraciones, cada una responsable de uno o dos de los servicios exigidos por el proyecto y ejecutando dentro de sí las fases de análisis, diseño, implementación y pruebas: la Iteración 1 cubre la infraestructura Grid base, el directorio activo y la configuración del clúster HPC; la Iteración 2 cubre los servicios File Sync y Shared File (RMI); la Iteración 3 cubre los servicios Photo Album y Streaming (SOAP); y la Iteración 4 cubre el servicio MPI Jobs (gRPC) y el módulo de monitoreo. Cada iteración finaliza con una revisión del registro de riesgos del proyecto, permitiendo ajustar el plan de contingencia de forma temprana, y con una integración incremental de los servicios ya construidos sobre la versión anterior del sistema, evitando una integración única al final del proyecto. La sección 7 detalla el cronograma resultante de esta planeación.

6 Resultados

[Sección en construcción]. A la fecha de esta versión del informe (semana 1 del periodo 2026-20), el proyecto se encuentra en su fase de planificación, por lo que aún no existen resultados de implementación que reportar. Esta sección se completará progresivamente a medida que avance el desarrollo, siguiendo los hitos definidos en el cronograma de la sección 7. Se documentarán aquí, entrega tras entrega: la arquitectura final implementada, las pruebas funcionales realizadas sobre cada uno de los cinco servicios (File Sync, Shared File, Photo Album, Streaming y MPI Jobs), las métricas de desempeño del clúster HPC recolectadas por el módulo de monitoreo, y la evidencia de cumplimiento de los requisitos de seguridad informática exigidos por el proyecto.

7 Cronograma

La Tabla 1 presenta el cronograma planeado del proyecto para las 17 semanas del periodo académico 2026-20, calculado a partir del inicio de clases el 14 de julio de 2026. Este cronograma corresponde a la planeación inicial del equipo y deberá actualizarse en la versión final del artículo para reflejar el cronograma efectivamente ejecutado, conforme lo exige el formato de entrega del curso.

Tabla 1. Cronograma planeado del proyecto.

Sem.	Fechas 2026	Fase	Hito principal
1	14 – 20 jul	Planificación	Roles del equipo, alcance técnico y tecnologías validadas.
2	21 – 27 jul	Planificación / Diseño	Arquitectura general y contratos de API entre servicios.
3	28 jul – 3 ago	Diseño	Modelo de directorio activo, permisos Unix y esquema de seguridad.
4	4 – 10 ago	Iteración 1	Infraestructura Grid base y directorio activo desplegados.
5	11 – 17 ago	Iteración 1	Clúster HPC configurado; primeras pruebas de ejecución MPI.
6	18 – 24 ago	Iteración 2	Servicio File Sync funcional.
7	25 – 31 ago	Iteración 2	Servicio Shared File (RMI) funcional.
8	1 – 7 sep	Cierre de Iteración	Primer informe de seguimiento (bitácora).
9	8 – 14 sep	Iteración 3	Servicio Photo Album desplegado.
10	15 – 21 sep	Iteración 3	Servicio Streaming (SOAP) funcional.
11	22 – 28 sep	Iteración 4	Servicio MPI Jobs (gRPC) integrado con el clúster.
12	29 sep – 5 oct	Iteración 4	Módulo de monitoreo desplegado.
13	6 – 12 oct	Integración	Integración completa y pruebas end-to-end.
14	13 – 19 oct	Producto	Entrega del producto funcional completo.
15	20 – 26 oct	Participación institucional	Evento obligatorio (20-22 oct.) y hardening de seguridad.
16	27 oct – 2 nov	Cierre	Segundo informe, artículo científico final y Presentación JSPI.
17	3 – 9 nov	Sustentación	Sustentación de carácter individual.

8 Discusión y conclusiones

[Sección en construcción]. Dado que este informe se presenta en la fase inicial del proyecto, la discusión completa de resultados y las conclusiones definitivas se desarrollarán una vez se cuente con el producto integrado (semana 14) y las métricas del módulo de monitoreo. A partir del estado del arte revisado en la sección 2, se anticipan como principales retos del proyecto la integración oportuna entre los cinco servicios desarrollados por sub-equipos distintos y el cumplimiento simultáneo de los requisitos de seguridad informática sin sacrificar el desempeño del clúster HPC, ambos identificados como riesgos críticos en el acta de constitución del proyecto.

Como contribución esperada, este trabajo busca documentar un caso aplicado de integración entre múltiples paradigmas de sistemas distribuidos —objetos distribuidos, servicios web, paso de mensajes y computación en malla— dentro de una única solución

coherente, que pueda servir de referencia para futuros desarrollos de infraestructura científica dentro de la UPB seccional Bucaramanga.

Acknowledgments. Los autores agradecen al docente MSc. Lenin Javier Serrano Gil por la orientación brindada durante el desarrollo de este proyecto, así como a la Facultad de Ingeniería de Sistemas e Informática de la UPB seccional Bucaramanga por el acceso al Centro de Computación Avanzada.

Disclosure of Interests. Los autores declaran no tener conflictos de interés relacionados con el contenido de este artículo.

Referencias bibliográficas

1. Adepoju, S., & Ologunagba, G. F. (2025). A systematic review and comparative evolution of grid computing and modern cloud computing infrastructures. International Journal of Scientific Research in Computer Science and Engineering, 13(5), 43–51.
2. AlHafez, N., & Kurdi, A. (2025). Parallel paradigms in modern HPC: A comparative analysis of MPI, OpenMP, and CUDA. arXiv preprint arXiv:2506.15454.
3. Babu, R., Abraham, G., & Borasia, K. (2013). A review on securing distributed systems using symmetric key cryptography. arXiv preprint arXiv:1303.0351.
4. Capra, R. (2014). File synchronization and sharing: User practices and challenges. Proceedings of the American Society for Information Science and Technology, 51(1), 1–10.
5. Castillo, P. A., Bernier, J. L., Arenas, M. G., Merelo, J. J., & García-Sánchez, P. (2011). SOAP vs REST: Comparing a master-slave GA implementation. arXiv preprint arXiv:1105.4978.
6. Guerraoui, R., & Schiper, A. (1996). Fault-tolerance by replication in distributed systems. En A. Strohmeier (Ed.), Reliable Software Technologies — Ada-Europe ’96 (Lecture Notes in Computer Science, vol. 1088, pp. 38–57). Springer.
7. Juric, M. B. (2000). Comparison of CORBA and Java RMI based on performance analysis. Actas de los Workshops de ECOOP.
8. Kunz, P. (2022). HPC job-monitoring with SLURM, Prometheus and Grafana (Tesis de pregrado). Universidad de Basilea.
9. Mehdi, Z., & Ragab-Hassen, H. (2016). File synchronization systems survey. Actas de ICCSEA, SPPR, UBIC 2016, 29–38.
10. Munoz, C., & Zalewski, J. (2001). Architecture and performance of Java-based distributed object models: CORBA vs RMI. Real-Time Systems, 21(1).
11. Natali, L., & Merani, M. L. (2015). Adaptive streaming in P2P live video systems: A distributed rate control approach. arXiv preprint arXiv:1510.04472.
12. Priyadarshini, S. B. B. (2022). Security in distributed system: A review perspective.
13. Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum: La guía definitiva de Scrum: Las reglas del juego. scrumguides.org.
14. Sungkar, A., & Kogoya, T. (2020). A review of grid computing. Computer Science & IT Research Journal, 1(1), 1–6.
15. UPB. (2020). Carta descriptiva del curso: Sistemas distribuidos. Código DA-FO-085N, versión 3_3.
16. Waheed, M. H. B., Jamil, F., Qayyum, A., Jamil, H., Cheikhrouhou, O., Ibrahim, M., Bhushan, B., & Hmam, H. (2021). A new efficient architecture for adaptive bit-rate video streaming. Sustainability, 13(8), 4541.

17. Woo, T. Y. C., & Lam, S. S. (1992). Authentication for distributed systems. IEEE Computer, 25(1), 39–51.